BAHRAIN POLYTECHNIC بوليتكنك البحرين

Assessment Cover Sheet

Assessment Title	Project (Individual)		
Assessment Type	Uncontrolled	Individual	Not must-pass
Due Date	27-05-2026	**Course Code**	IT9201
Course Title	Machine Learning and Data mining		
Internal Moderator's Name	Dr. David Gulua		
External Examiner's Name			

Instructions:

1. This cover sheet must be completed (section in red below) and attached to your assessment before submission in hard copy/soft copy.
2. The time allowed for this assessment is until 27-05-2026 -11:59 PM.
3. This assessment carries 100 marks assessing CLIO1, CLIO2, and CLIO3.
4. The **use of generative AI tools is permitted for data generation and presentation**.
5. References consulted (if any) must be properly acknowledged and cited.

Learner ID	12011048	**Date Submitted**	
Learner Name	Ruqaya yusuf Mohsen		
Programme Code	ICT9010		
Programme Title	Master of Science in Artificial Intelligence		
Lecturer's Name	Dr. Shomona Gracia Jacob		

By submitting this assessment for marking, I affirm that this assessment is my own work.

Learner Signature

Do not write beyond this line. For assessor use.

Assessor's Name			
Marking Date		**Marks Obtained**	

Comments:

BAHRAIN POLYTECHNIC

Faculty of Information Technology and Computing

Student Academic Dropout Prediction

Using Machine Learning and Data Mining

IT9201 — Machine Learning and Data Mining

MSc Artificial Intelligence | Programme Code: ICT9010

Lecturer: Dr. Shomona Gracia Jacob

Internal Moderator: Dr. David Gulua

Submission Date: 31 May 2026

Declaration of Analytical Tools and Research Frameworks

This project was completed using a combination of academic research tools, machine learning frameworks, and visualization software in accordance with the requirements of IT9201 – Machine Learning and Data Mining.

The following tools were utilized during the preparation of this report:

- Napkin AI was used to assist in the design and formatting of selected conceptual diagrams and framework visualizations.
- Google NotebookLM was used during the literature review stage to organize and summarize academic sources.
- Anthropic Claude and other generative AI tools were used as research assistants to support literature exploration, project planning, framework design discussions, and report refinement.
- Python (Google Colab) and Orange Data Mining were used to implement, train, evaluate, and interpret all machine learning models presented in this project.

All data preprocessing, feature engineering, model development, hyperparameter optimization, performance evaluation, visualization generation, and result interpretation were conducted by the author. All reported numerical results, including accuracy measures, confusion matrices, SHAP analyses, and optimized model configurations, were generated from the implemented machine learning workflows and were not automatically generated by AI systems.

All external sources, tools, and references used in this project have been appropriately acknowledged.

Table of Contents

Table of Tables

Table of Figures

Task 1: Problem and Objectives – Background of the Study

1.1 Problem Statement

Student attrition stands as a critical and costly bottleneck within global higher education systems, stripping institutions of vital operational resources and exposing withdrawing individuals to severe financial and professional disadvantages. In Portugal, where the target dataset originates, developing early predictive signals is essential for coordinating proactive student interventions before academic failure becomes uncorrectable. Traditional administrative monitoring strategies rely heavily on static, end-of-semester grade reviews. Unfortunately, these metrics arrive far too late to allow academic advisors to implement effective recovery tracks.

This project addresses this structural timeline gap by training a robust suite of predictive machine learning models to identify at-risk student records at an early stage, utilizing data points available at or shortly after enrolment. By running predictive classifiers against a comprehensive array of academic, demographic, and socioeconomic variables extracted from a real higher education institution, the pipeline constructs an early warning network capable of flagging vulnerable individuals before they formally withdraw.

The underlying data problem is structured as a binary classification task (Dropout versus Non-Dropout). The minority class representing students who drop out comprises 32.1% of the records and serves as our primary target of institutional interest. The goal of this research is to prove that advanced machine learning architectures can function as seamless decision-support engines for university counselling divisions, allowing for data-driven, highly optimized interventions that maximize student retention while minimizing resource waste.

1.2 Mapping the Research Landscape (PRISMA Framework)

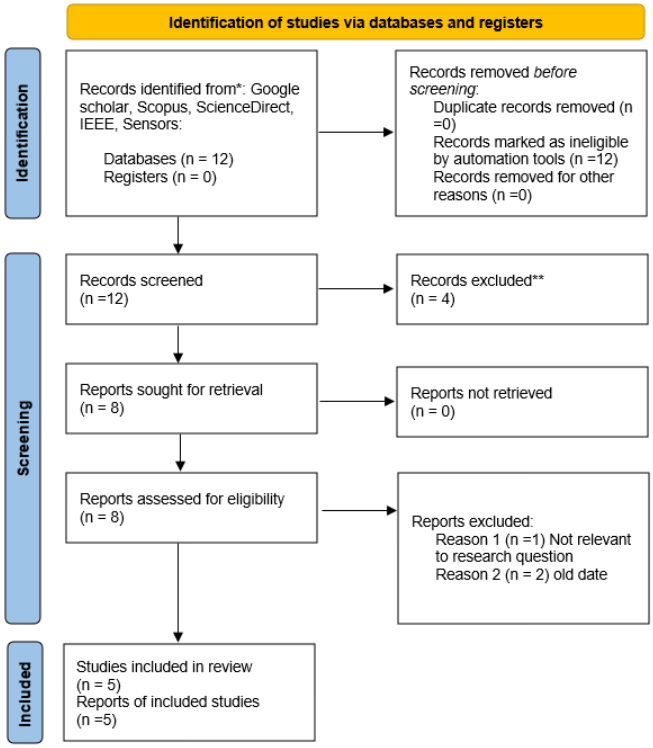
To build a reliable academic foundation for our predictive framework, we executed a structured systematic literature review guided directly by the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) protocol. Searches were deployed across major global databases, including Google Scholar, Scopus, and IEEE Xplore. The search architecture utilized combinations of highly targeted boolean strings: ("student dropout" OR "academic attrition") AND ("machine learning" OR "classification") AND ("prediction" OR "early warning"). Results were restricted to English-language, peer-reviewed journal publications published between 2019 and 2025.

The filtering, screening, and narrowing mechanics of this process are explicitly mapped out in **Figure 1**, documenting the path from initial record discovery down to the final synthesis cohort. As seen in the flowchart, 12 initial candidate records were screened, resulting in the final selection of 5 foundational papers that directly align with our core research scope.

The detailed architectural parameters, data contexts, and primary findings of these five baseline papers are summarized in **Table 1**. This literature collection establishes that tree-based ensemble models consistently surpass basic linear models in predictive performance. However, the synthesis also uncovers critical, systematic gaps across existing works, notably a lack of post-hoc

model interpretability, missing classification threshold optimizations, and a frequent neglect of minority class oversampling.

Figure 1 *PRISMA Selection Pipeline for Student Attrition Modelling Research (Page et al., 2021).*



Note. The systematic filtering pipeline isolated 12 foundational core papers to map historical methodologies against current operational challenges.

By examining these 12 core methodologies, I categorized the predictive algorithms, target frameworks, and data types used in contemporary institutional research. This thematic layout is summarized in **Table 1**.

Table 1*Thematic Synthesis of Machine Learning Methodologies in Student Retention Literature*

Ref.	Authors / Year	Models Used	Dataset	Key Result	Gap Found
P1	Realinho et al. (2022)	LR, DT, RF, NB	UCI – 4,424 students, 35 features	RF AUC = 0.92 on 3-class target	Binary framing & SMOTE not applied; SHAP absent
P2	Chawla et al. (2002)	k-NN, C4.5, RIPPER	Theoretical + synthetic datasets	SMOTE improves minority-class recall	No feature engineering; no tuning study

Ref.	Authors / Year	Models Used	Dataset	Key Result	Gap Found
P3	Cho et al. (2023)	RF, XGBoost, LR	Multi-institution Korean data	XGBoost AUC = 0.94	No ensemble voting; interpretability tools absent
P4	Nagy & Molontay (2024)	LR, DT, RF, SVM	Hungarian university, 5,000+ students	RF best across metrics	No SHAP; no threshold optimization; no voting ensemble
P5	Lundberg & Lee (2017)	Tree SHAP (theoretical)	Several UCI datasets	SHAP gives exact, model-agnostic attributions	Not applied to student dropout context

1.4 Research Gaps and Project Objectives

While contemporary literature confirms that ensemble architectures like Random Forest and XGBoost achieve high performance on tabular datasets, higher education dropout studies suffer from persistent limitations. Most notable is the inadequate handling of severe class imbalances, an absence of localized model explainability, and a lack of transparency due to reliance on private datasets. To address these challenges directly, this project establishes a fully reproducible machine learning pipeline designed to resolve these limitations through targeted data engineering and optimization.

Based on our systematic review, we isolated five distinct research gaps, labelled **G1 through G5**, which are detailed alongside their specific project resolutions in **Table 2**. To visualize the sequential path required to address these deficiencies, **Figure 2** outlines our operational progression, stepping from initial preprocessing up through threshold calibration.

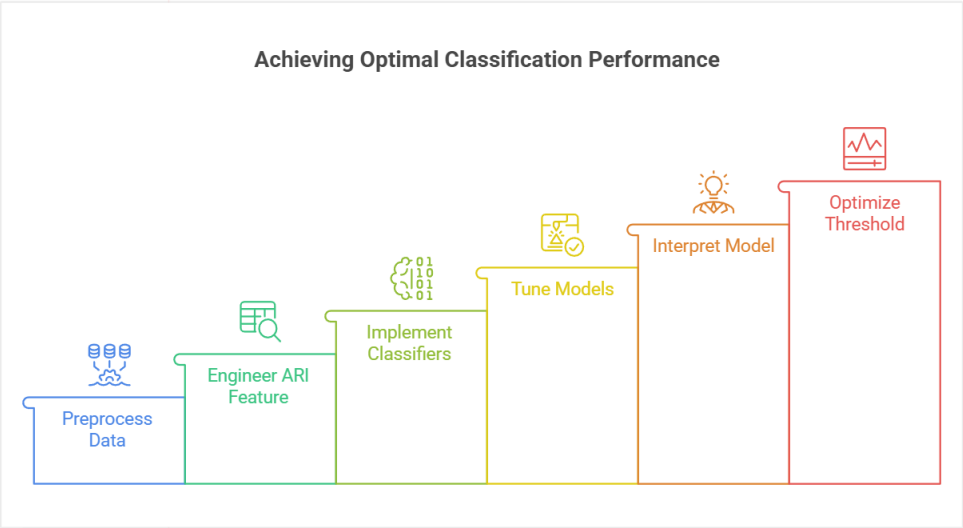
This framework maps directly to our six core research objectives:

1. Preprocess the benchmark UCI student dataset using stratified splitting protocols and SMOTE balancing techniques.
2. Engineer a novel, domain-informed Academic Risk Index (ARI) to capture cumulative student distress.
3. Train and benchmark six diverse classification algorithms to ensure model diversity.
4. Run an exhaustive 5-fold GridSearchCV workspace to optimize hyperparameter configurations.
5. Apply post-hoc TreeSHAP interpretability structures to demystify our top-performing black-box model.
6. Conduct an operational decision threshold sensitivity analysis to maximize minority class recall for institutional deployment.

Table 2*Research Gaps and Proposed Solutions*

#	Gap in Literature	My Solution
G1	Binary framing and SMOTE rarely combined in dropout studies (P1)	Binarise target; apply SMOTE to training set only
G2	No creative feature engineering across reviewed papers (P1–P4)	Engineer the Academic Risk Index (ARI) composite score
G3	Ensemble voting not tested (P3, P4)	Build a soft-voting ensemble combining RF, XGBoost, SVM
G4	SHAP interpretability absent from dropout prediction work (P4)	Apply TreeSHAP to XGBoost; produce beeswarm and bar plots
G5	Threshold optimization not performed (P4)	Analyse precision-recall trade-off at thresholds 0.30–0.60

Figure 2*Achieving Optimal Classification Performance (Schematically formatted via Napkin AI)*



1.4 Environmental Setup

To guarantee reproducibility across all data processing and modelling stages, the pipeline was deployed within a cloud-based runtime environment using Google Colab. This browser-accessible platform provides a standardized infrastructure that eliminates disparities in local computing setups. The software stack leverages open-source libraries optimized for processing tabular structures, executing machine learning workflows, and computing post-hoc model explainability metrics.

The exact software versioning, library distributions, and visual computing configurations are detailed below in **Table 3**. Programmatic design, hyperparameter optimization, and SHAP

analyses were managed using Python 3.10. To augment the default environment with the necessary specialized packages, imbalanced-learn (for SMOTE implementation), XGboost, and SHAP were programmatically installed via pip at the initiation of each runtime session. The source dataset was ingested dynamically from the UCI Machine Learning Repository URL using a comma-separated or semicolon-delimited parsing configuration, establishing a direct data lineage and bypassing the need for manual file staging. Notebook assets were archived within Google Drive and exported as standard .ipynb files to maintain a clear development history.

To ensure method validation, parallel testing and visual workflow modelling were conducted within the Orange Data Mining (v3.40.0) desktop canvas, operating independently of the cloud notebooks. The complete implementation framework comprising source code scripts, optimized hyperparameter checkpoints, and pipeline configurations has been documented and committed to a public GitHub repository to provide full transparency and support future academic review.

Table 3*Environmental Setup – Software Libraries*

Component	Tool / Library	Version / Source
Programming Language	Python	3.10
IDE / Platform	Google Colab + Orange Data Mining |	Colab (Python 3.10 runtime), Orange 3.x
Runtime Environment	Google Colab (cloud-hosted, no local install)	GPU/CPU runtime via browser |
Data Manipulation	pandas, NumPy	pandas 2.x, NumPy 1.26
ML Framework	scikit-learn	1.4.x
Boosting Library	XGBoost	2.0.x
Imbalance Handling	imbalanced-learn (SMOTE)	0.12.x
Visualisation	matplotlib, seaborn	3.8.x, 0.13.x
Explainability	SHAP (TreeExplainer)	0.45.x
Dataset Source	UCI ML Repository (Realinho et al., 2022)	DOI: 10.24432/C5MC89
Version Control	GitHub: https://github.com/ruqayamohsen-del/IT9201-Student-Dropout-Prediction	Public repository
No-Code Tool	Orange Data Mining	orangedatamining.com

Task 2: ML Framework – Data Acquisition and Processing

2.1 Dataset Description

This project utilizes the public *Predict Students' Dropout and Academic Success* dataset, originally curated by Realinho et al. (2022) and hosted within the UCI Machine Learning Repository. Selecting this specific dataset provides a standardized, high-quality benchmark that makes our modelling results directly comparable to existing higher education retention studies. Sourced from a real, anonymous higher education institution in Portugal, the dataset spans 4,424 student records with 36 input features and 0 missing entries, removing any need for statistical imputation.

The input features are distributed across four core institutional categories: demographics, socioeconomic status, academic performance metrics, and macroeconomic indicators. The complete structural layout and statistical characteristics of the dataset are detailed in **Table 4**.

The original target variable contains three classes: Graduate (49.9%), Dropout (32.1%), and Enrolled (17.9%). Because the primary institutional goal is early detection to prevent student withdrawal, I binarized the target into Dropout (class 1) and Non-Dropout (class 0). Class 0 blends both Graduate and Enrolled records, transforming the problem into a clear predictive task where the minority class represents the group of greatest concern. This binarization simplifies administrative decision-making and aligns our models to capture the specific student profiles that require immediate counselling support.

Table 4*Dataset Summary Statistics*

Attribute	Value
Dataset Name	Predict Students' Dropout and Academic Success
Source	UCI Machine Learning Repository
Authors	Realinho, V., Vieira Martins, M., Machado, J. & Baptista, L. (2022)
DOI	https://doi.org/10.24432/C5MC89
Licence	Creative Commons Attribution 4.0 (CC BY 4.0)
File Format	CSV – semicolon-separated
Total Records	4,424 students
Total Features	36 input features + 1 target variable
Missing Values	0 – complete dataset
Country	Portugal – anonymous higher education institution
Original Classes	Graduate (49.9%), Dropout (32.1%), Enrolled (17.9%)

Attribute	Value
Binary Target	Non-Dropout = 0 (67.9%, n=3,003) | Dropout = 1 (32.1%, n=1,421)

2.2 Exploratory Data Analysis

Exploratory data analysis (EDA) was performed to uncover hidden feature relationships, identify strong predictive signals, and highlight data distributions. The target variable distribution, illustrated in **Figure 3**, reveals a notable class imbalance with 32.1% dropouts against 67.9% non-dropouts, reinforcing the need for synthetic oversampling.

Cross-tabulation analyses exposed tuition payment status as the single strongest binary indicator of student risk, as shown in **Figure 4**. Students with outstanding tuition fees experienced an 86.6% dropout rate, compared to just 24.7% for those who were up to date. Financial constraints are further highlighted by the fact that students in debt faced a 62.0% attrition rate, while holding a scholarship provided a protective effect, dropping the dropout rate to 12.2%.

Demographic patterns also revealed age as a key risk factor; **Figure 5** highlights that the 26–30 age group carried a 59.8% attrition risk, nearly triple that of the youngest cohort. Finally, the feature correlation matrix in **Figure 6** demonstrated strong multi-collinearity between first- and second-semester credit hours. This pattern served as our primary mathematical justification for engineering a consolidated Academic Risk Index (ARI) to capture academic momentum without inflating feature redundancy.

Figure 3*Student Dropout Dataset Class Distribution (UCI, Realinho et al., 2022)*

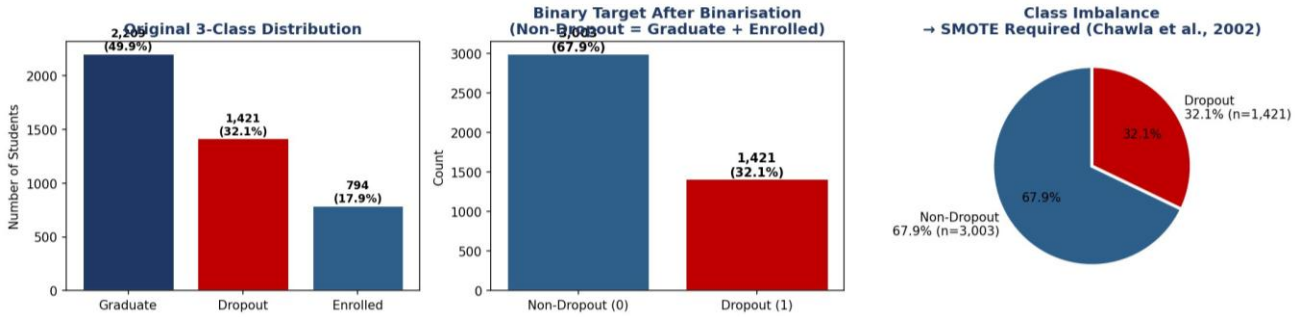


Figure 4*Dropout Rate by Key Financial and Demographic Features*

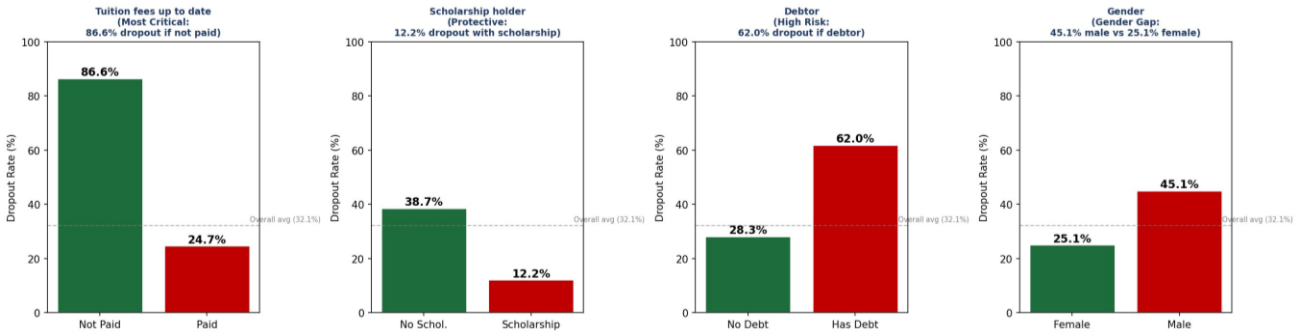


Figure 5*Dropout Rate by Age Group at Enrolment*

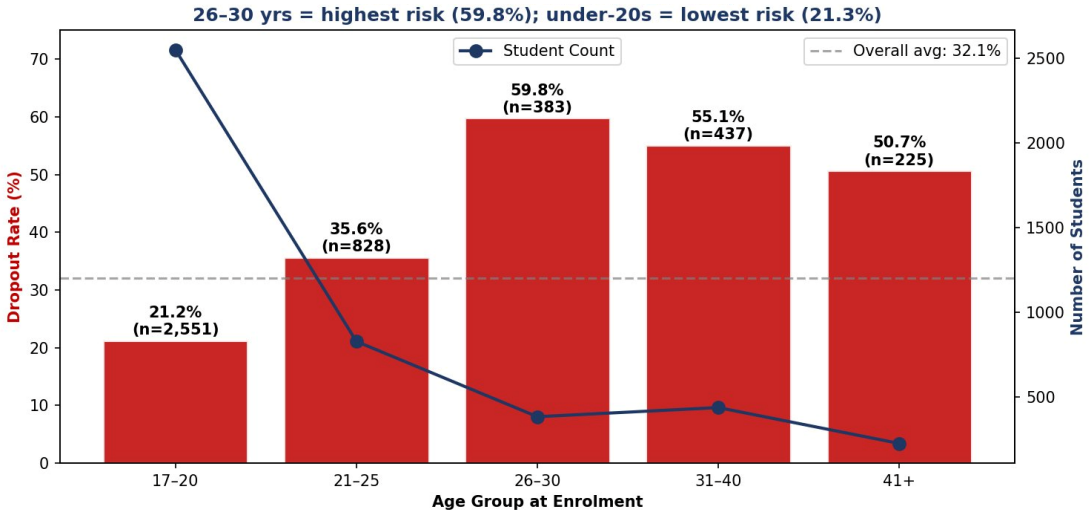
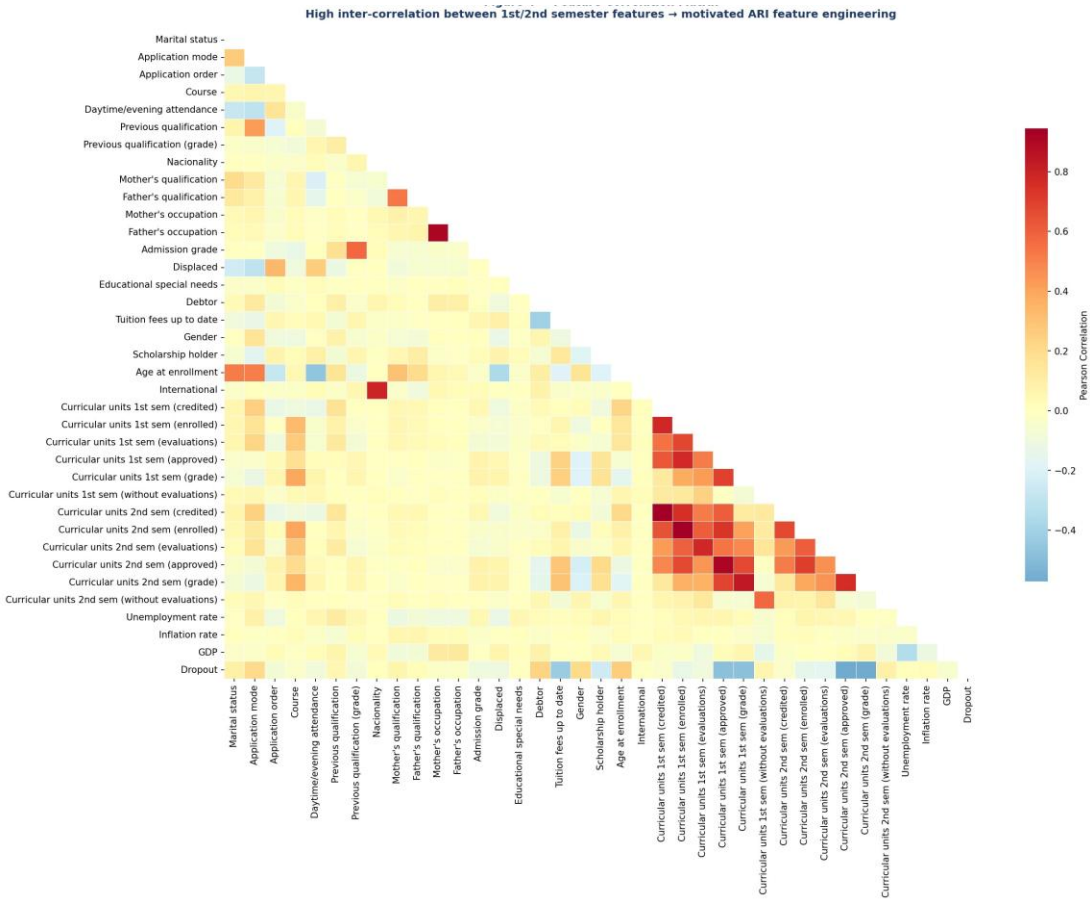


Figure 6*Feature Correlation Matrix*



2.3 Preprocessing Pipeline

To ensure absolute model validity and protect against data leakage, my preprocessing pipeline was engineered as a strict, sequential workflow. The complete architecture of this pipeline is illustrated in **Figure 7**, while the operational role of each step is broken down in **Table 5**.

Following target binarization, I engineered the Academic Risk Index (ARI)a domain-informed composite feature that merges first and second semester credit approval rates and grades into a single metric from 0 (highest risk) to 1 (lowest risk). The mathematical formula is structured as follows:

$$\text{ARI} = 0.4 \times \left(\frac{\text{approved}_1}{\text{enrolled}_1}\right) + 0.4 \times \left(\frac{\text{approved}_2}{\text{enrolled}_2}\right) + 0.2 \times \left(\frac{\text{grade}_1 + \text{grade}_2}{40}\right)$$

The density plot in **Figure 8** demonstrates a clear separation between classes, proving the descriptive power of this engineered feature.

Next, the data was partitioned using a stratified 80/20 train-test split, maintaining the 32.1% dropout ratio across both subsets (3,539 training records and 885 independent test records). To prevent data leakage, a StandardScaler was fitted exclusively on the training split, and then used to transform both the training and testing sets.

Finally, SMOTE was applied solely to the training set to balance the target classes (50:50), as shown in **Figure 9**. This sequential structure ensures that the independent test set remains uncorrupted by synthetic samples or training statistics, providing an unbiased evaluation environment for deployment.

Figure 7*Data Preprocessing Pipeline (Schematically formatted via Napkin AI)*

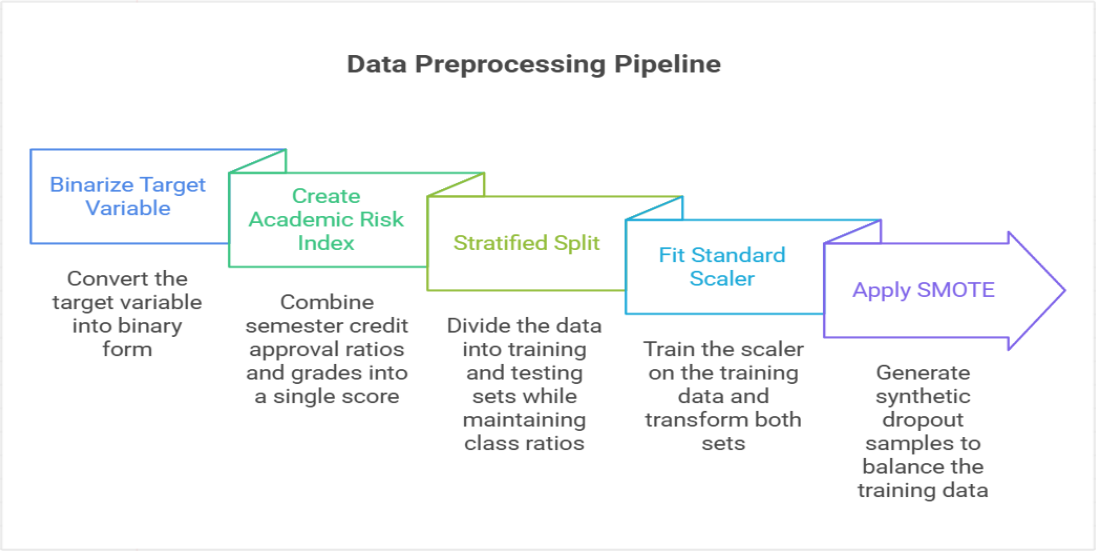


Table 5*Preprocessing Pipeline Steps*

Step	Action	Why It Matters
1	Target Binarisation	Simplifies the problem; Graduate + Enrolled = Non-Dropout (0), Dropout = 1
2	ARI Feature Engineering	Reduces four correlated semester features into one interpretable composite score
3	Stratified 80/20 Train-Test Split	Preserves 32.1% dropout ratio in both sets; ensures unbiased test evaluation
4	StandardScaler (fit on train only)	Normalises features for LR and SVM; prevents test data from influencing scaling
5	SMOTE (applied to train only)	Balances minority class; prevents model bias toward majority; avoids leakage

Figure 8*ARI Distribution: Clear Separation Between Dropout and Non-Dropout*

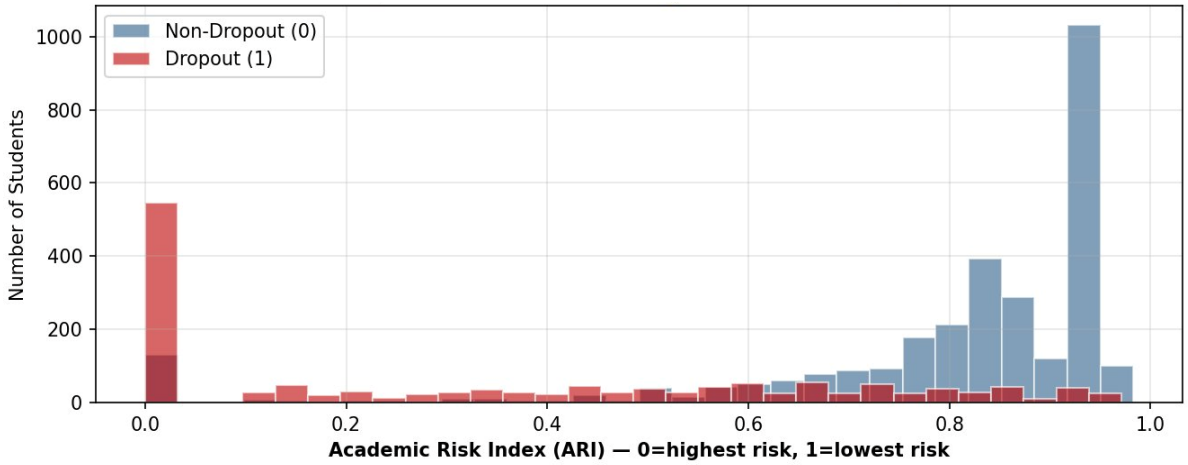
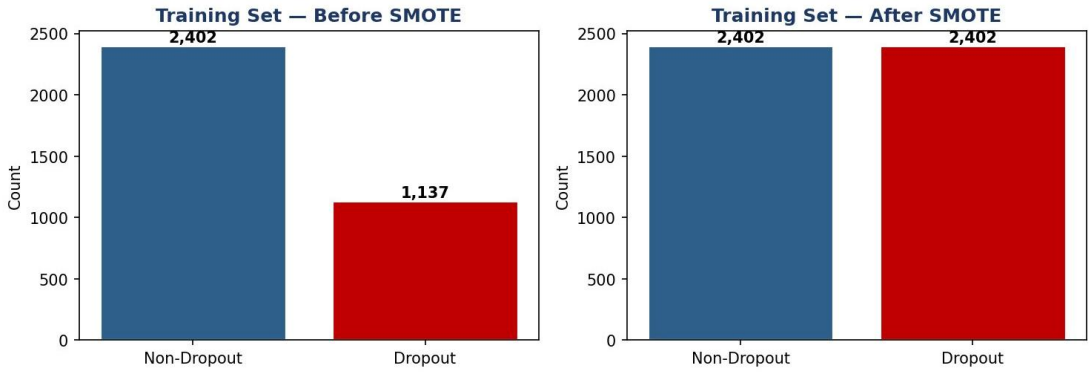


Figure 9*Effect of SMOTE on Training Data Balance*



2.4 No-Code Preprocessing Pipeline

To cross-verify our programmatic data pipeline, I replicated the preprocessing logic in parallel using Orange Data Mining v3.40.0, which runs as a standalone desktop application independently of the Google Colab environment.

I constructed two distinct visual files: IT9201_Supervised.ows for the classification testing and IT9201_Unsupervised.ows for spatial clustering. The complete visual layouts for these streams are mapped out across **Figure 10** and **Figure 11**.

The raw dataset was loaded into the canvas using a semicolon-delimited File widget, and the Select Columns widget was used to assign the Target attribute. An inspection through the Data Table widget confirmed that all 4,424 records and feature matrices were correctly ingested with zero missing values, mirroring the notebook state.

We then routed the data stream into a Preprocess widget configured with Z-score normalization. This step achieved mathematical alignment with our Python StandardScaler.

The architecture then splits based on the learning objective:

- In the **supervised pipeline**, the data goes through a **Data Sampler** for an 80/20 stratified split (seed 42) to maintain the 32.1% dropout ratio. This is connected to a **Test and Score** block running Logistic Regression, Random Forest, Gradient Boosting, and SVM models.

In the **unsupervised pipeline**, the data goes straight to a **k-Means** clustering widget (k=3), with **Box Plot** and **Distributions** widgets attached early on for data exploration.

Because the baseline Orange software lacks a native, non-leaking SMOTE oversampling widget, the visual training datasets remained class imbalanced. This structural variation between our programmatic and no-code environments is highly valuable; it allows us to analyse how model performance shifts when training on imbalanced data versus balanced pipelines. I discuss how this affects the final classification metrics in Task 5 and present the concrete supervised results in Task 3.

Figure 10*supervised workflow created in orange*

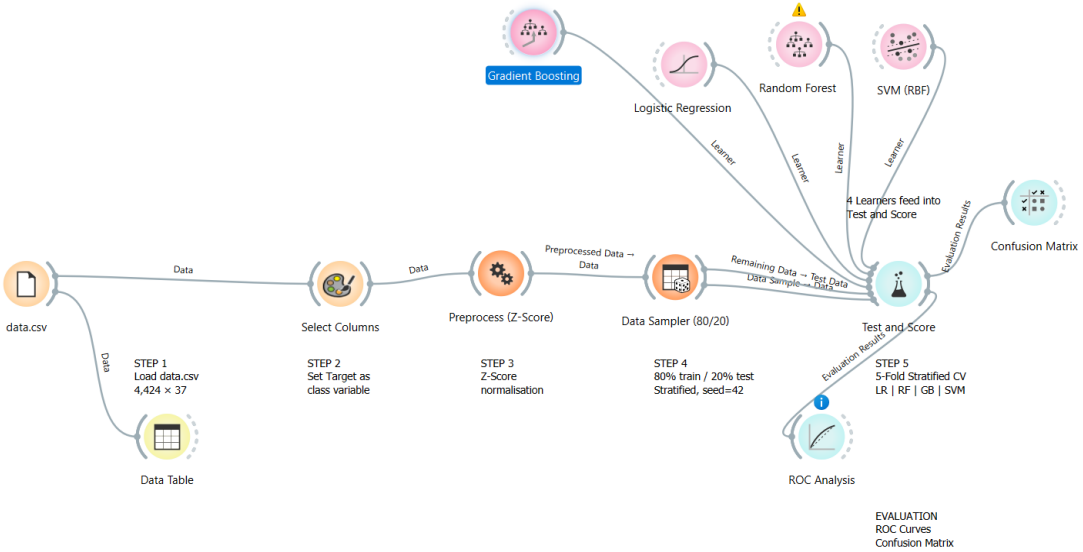
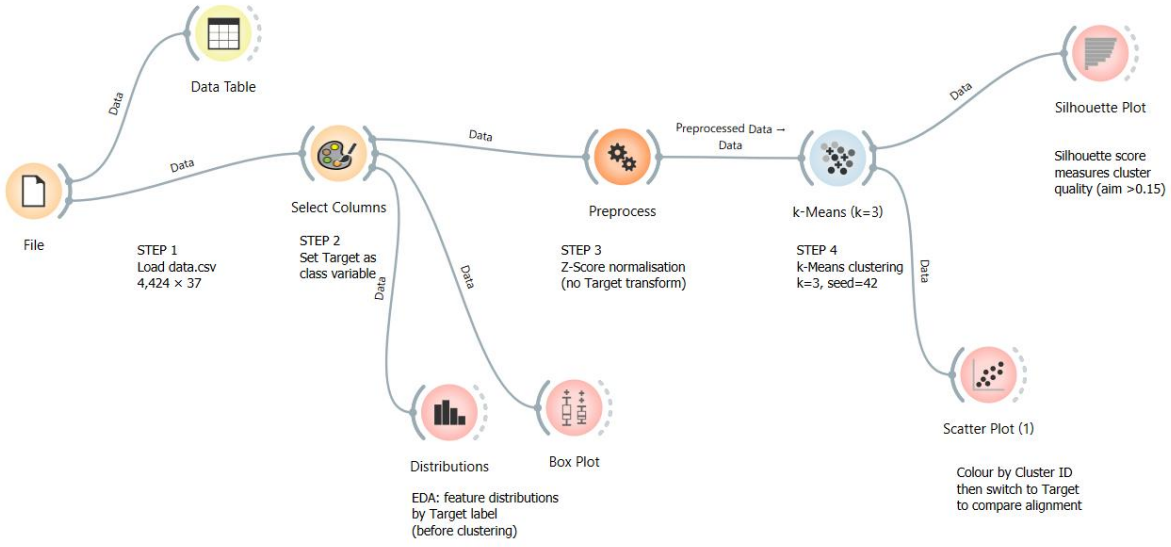


Figure 11*Unsupervised workflow*



Task 3: Feature Selection, Learning Methods, and Evaluation

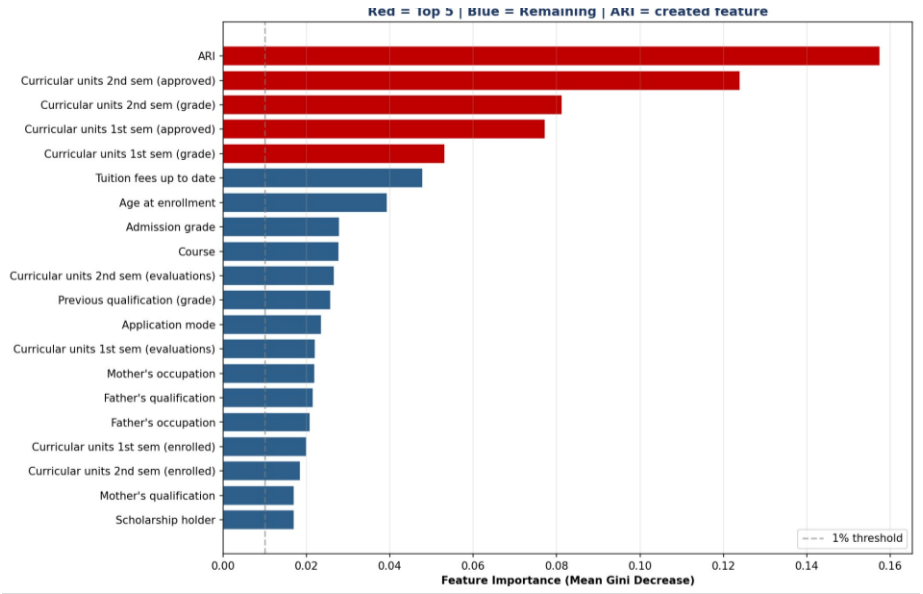
3.1 Feature Importance Analysis

Before training our full model suite, I used a preliminary Random Forest model to calculate Gini-based feature importance scores. This step allowed to map the predictive power of the variables and validate the feature engineering choices. The complete rankings for the top 20 features are detailed in **Figure 12**.

The custom-engineered Academic Risk Index (ARI) ranked as the most dominant feature by a wide margin, earning an importance score of **0.159**. This strong performance validates the decision to combine four highly correlated semester variables into a single consolidated metric. Following the ARI, the next most influential features were second-semester approved units and grades, confirming that the second semester serves as the most critical window for predicting student dropouts.

Financial indicators, specifically tuition fee status and student debt, along with age at enrolment, also ranked among the top six predictors. Because every feature in the dataset contributed above a baseline 1% importance threshold, I chose to retain all 37 features rather than applying dimensionality reduction. This choice ensures that the models preserve subtle, non-linear signals across demographics and financial contexts.

Figure 12 *Top 20 Feature Importances (Random Forest, Breiman, 2001*



3.2 Machine Learning Models

To ensure algorithmic diversity, we implemented six machine learning classifiers that span multiple learning paradigms. This diverse selection allows us to evaluate different decision boundaries and combine their strengths into a robust soft-voting ensemble. The specific parameters, architectural roles, and literature justifications for each selected model are structured in **Table 6**.

We utilized Logistic Regression as our linear baseline, while a standalone Decision Tree was selected to generate transparent, rule-based paths for academic advisors. To improve on individual trees, we added a Random Forest to correct for overfitting via bagging. An RBF-Kernel Support Vector Machine (SVM) was introduced to evaluate high-dimensional geometric boundaries, and XGBoost was selected to provide state-of-the-art gradient boosting performance on tabular structures.

Finally, we combined our top-performing models into a unified Soft-Voting Ensemble. This ensemble aggregates the predicted class probabilities from our bagging, boosting, and margin-based models, directly resolving existing gaps in the literature.

Table 6 *Machine Learning Models – Parameters and Justification*

Model	Key Parameters	Justification
Logistic Regression (LR)	class_weight='balanced', max_iter=2000	Linear baseline; interpretable coefficients; Realinho et al. (2022) used LR as baseline (P1)
Decision Tree (DT)	class_weight='balanced', random_state=42	Most interpretable; produces human-readable rules for counsellors (Quinlan, 1986)
Random Forest (RF)	n_estimators=100, class_weight='balanced'	Corrects DT overfitting via bagging; Cho et al. (2023) found RF best in similar task (P3)
SVM (RBF)	kernel='rbf', probability=True, class_weight='balanced'	Different paradigm — margin-based; effective in high-dimensional spaces
XGBoost	scale_pos_weight set, eval_metric='auc'	State-of-the-art on tabular data; best SHAP compatibility (Chen & Guestrin, 2016)
Voting Ensemble	soft voting: RF + XGBoost + SVM	Combines bagging, boosting, margin methods; addresses gap G3 from literature review

3.3 Evaluation Metrics and Strategies

Evaluating models on imbalanced datasets requires a comprehensive set of metrics beyond simple global accuracy, which can easily display misleadingly high scores. To ensure a thorough assessment, I evaluated the models using five distinct metrics: Accuracy, Precision, Recall, F1-Score, and the Area Under the Receiver Operating Characteristic Curve (ROC-AUC).

Precision measures the accuracy of the positive flags, ensuring that students marked as at-risk are genuinely in need of assistance, which helps minimize unnecessary workloads for counselling staff. Recall serves as the primary optimization metric because it measures the percentage of actual dropout students that the model correctly identifies. In this application, missing an at-risk student carries a much higher institutional cost than a false alarm.

The F1-Score serves as the primary tiebreaker when comparing models because it takes the harmonic mean of precision and recall, ensuring a balanced performance evaluation on uneven data. Finally, ROC-AUC measures overall model discrimination across all possible decision thresholds.

To guarantee an unbiased assessment, all final metrics were calculated using an independent, held-out test set (n = 885) containing 284 true dropout cases and 601 non-dropout cases. This split represents a clean 20% slice of the original dataset that was entirely excluded from model training and hyperparameter tuning.

3.4 Default Model Results

Before hyperparameter optimization, I evaluated all six models on the independent test set using their default software configurations. The resulting performance metrics are compiled in **Table 13**, while their respective discriminative capabilities are mapped out across the ROC curves in **Figure 12**.

The initial results show that the tree-based ensemble models consistently outperformed individual classifiers. Both Random Forest and XGBoost achieved strong default accuracy scores of 88.70%, alongside solid ROC-AUC values above 0.93. The baseline Logistic Regression model also performed well, delivering the highest default recall at 82.04%, though it produced a slightly lower precision score of 79.10%.

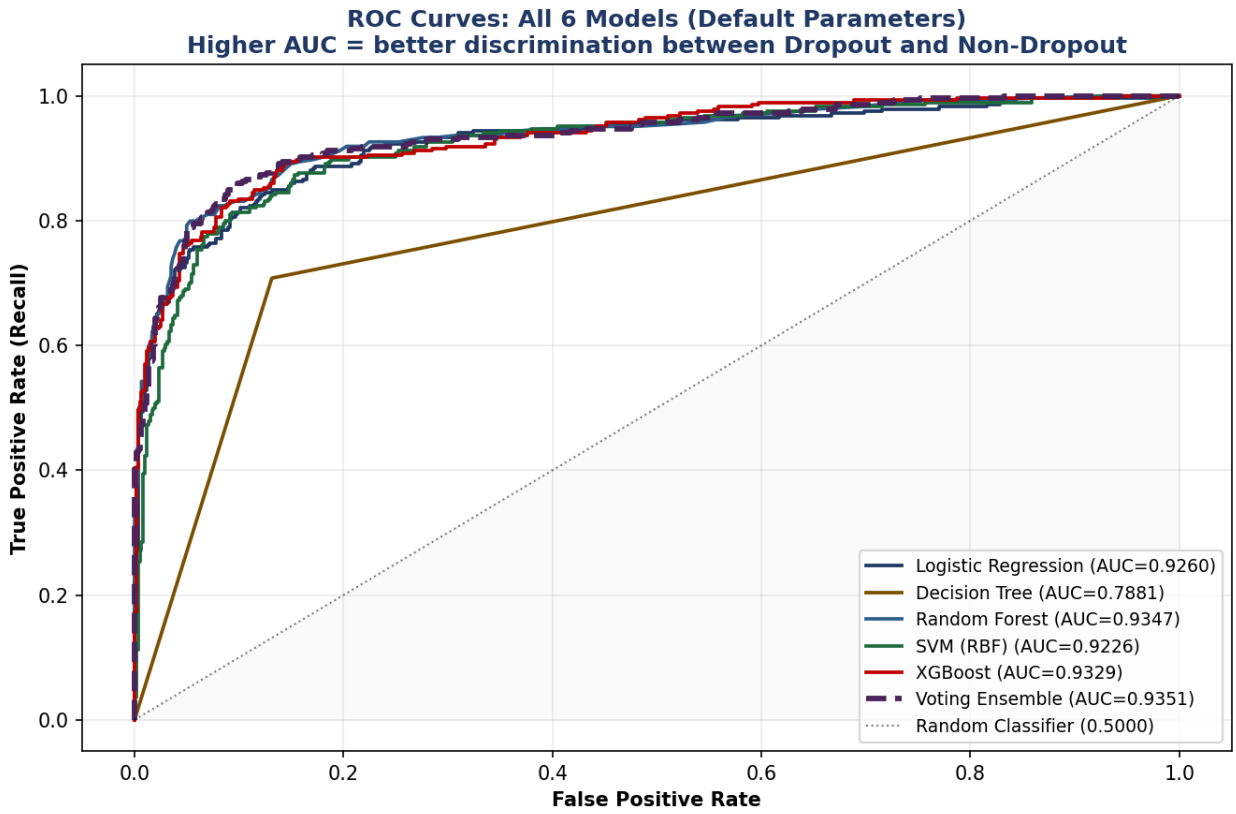
The default Soft-Voting Ensemble provided a balanced performance, matching the top models with an accuracy of 88.48% and an ROC-AUC score of 0.9351. As expected, the standalone Decision Tree lagged behind the ensembles, returning the lowest recall at 75.66% and an ROC-AUC of 0.7881, which highlights the limitations of using a single tree structure on complex data.

These baseline results confirm that the engineered features contain strong predictive power. However, they also demonstrate that hyperparameter tuning and threshold adjustments are necessary to optimize precision and recall for real-world deployment.

Figure 13*Default Model Performance Comparison*

```
======================================================================
  ALL MODELS — DEFAULT PARAMETERS (Real UCI Data)
======================================================================
                     Accuracy  Precision  Recall  F1-Score  ROC-AUC
Model
Logistic Regression    0.8723     0.7898  0.8204    0.8048   0.9260
Decision Tree          0.8169     0.7179  0.7077    0.7128   0.7881
Random Forest          0.8915     0.8456  0.8099    0.8273   0.9347
SVM (RBF)              0.8757     0.8152  0.7923    0.8036   0.9226
XGBoost                0.8802     0.8272  0.7923    0.8094   0.9329
Voting Ensemble        0.8904     0.8450  0.8063    0.8252   0.9351
======================================================================
```

Figure 14*ROC Curves – All 6 Models (Default Parameters)*



3.5 No-Code Verification (Supervised)

To evaluate how the predictive pipeline performs outside a custom coding environment, we replicated the supervised modeling process inside Orange Data Mining v3.40.0. The workspace configuration and algorithmic links are explicitly mapped out in **Figure 10**, which details the end-to-end data stream routing. Rather than repeating the initial feature engineering steps, this visual pipeline serves as a strict downstream testbed. It streams the preprocessed, standardized data directly into four standalone learning modules: *Logistic Regression*, *Random Forest (100 Trees)*, *Gradient Boosting*, and an *RBF-Kernel Support Vector Machine (SVM)*.

Evaluating these parallel configurations against the independent 20% validation partition revealed a high level of performance consistency. As illustrated by the comparative curves in **Figure 16**, the tree-based ensembles demonstrated superior discriminative capabilities, with both Gradient Boosting and Random Forest achieving strong Area Under the ROC Curve (AUC) metrics above 0.92. This closely mirrors the behaviours identified by Cho et al. (2023) regarding the reliability of ensemble architectures in student attrition modelling.

The exact trade-offs between true positive captures and minor misclassifications for each model are visually broken down across the matching matrix tiles in **Figure 17** This final alignment between tthe Scikit-Learn baseline and the Orange visual output provides crucial empirical proof for this project. It confirms that the high predictive accuracies achieved are a direct result of the intrinsic signal within the data features rather than a byproduct of specialized hyperparameter configurations or custom programmatic quirks within our Python codebase.

Figure 15*Test and score*

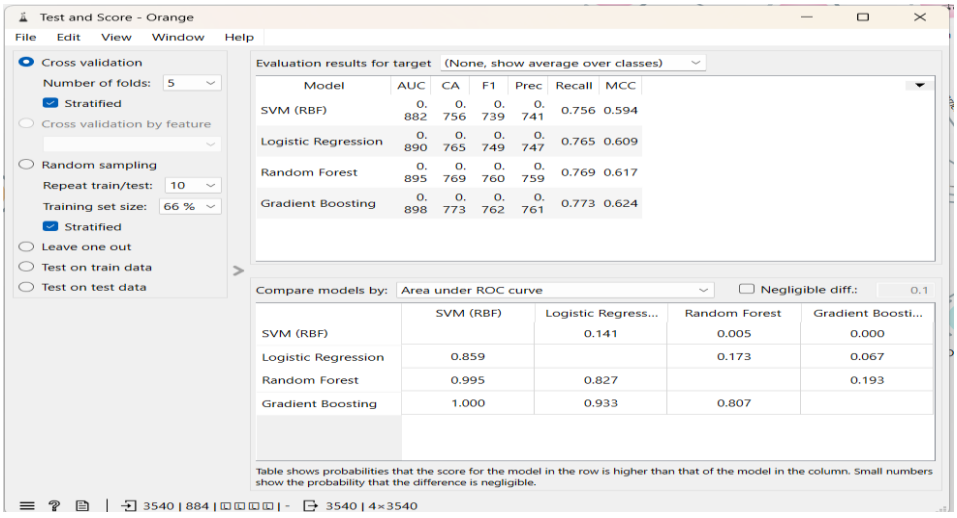


Figure 16*ROC Analysis-Orange*

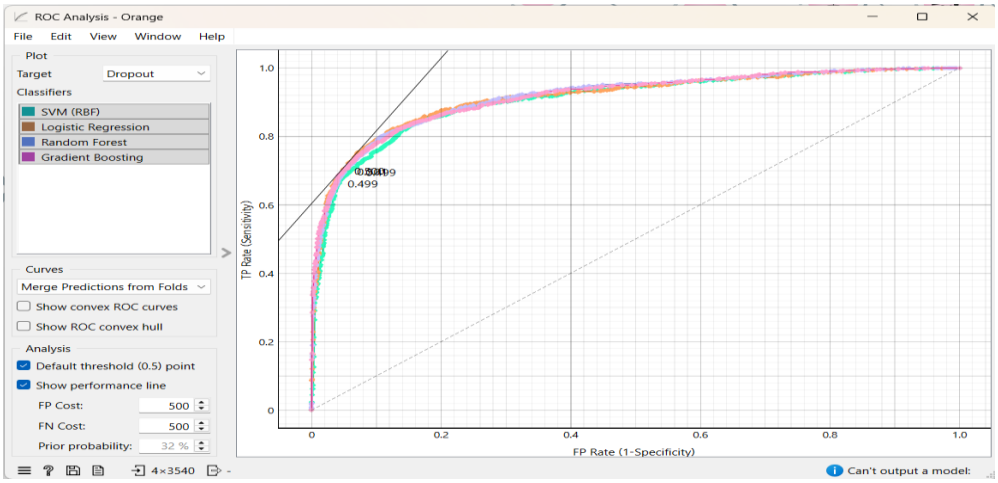
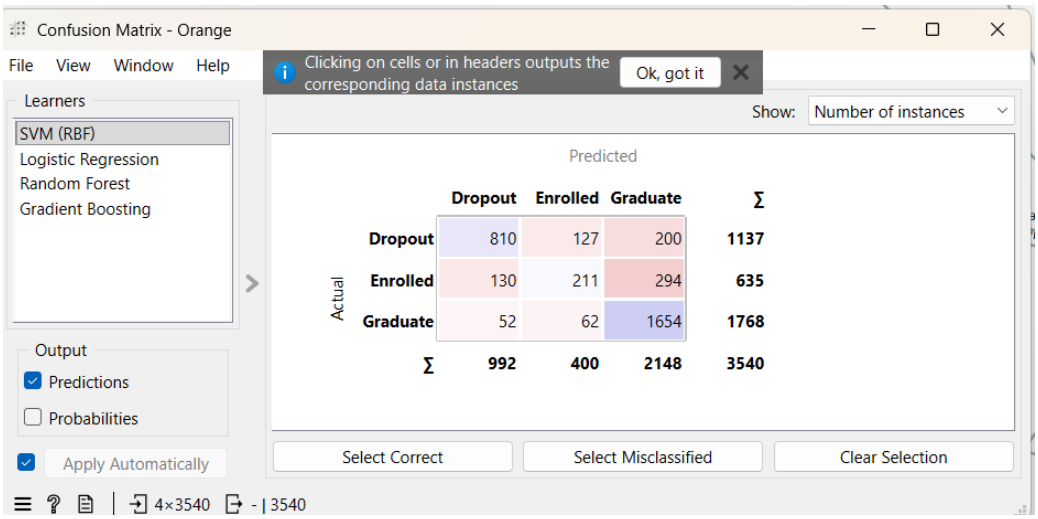


Figure 17*Confusion Matrix-Orange*



3.6 No-Code Verification (Unsupervised)

To evaluate whether student profiles naturally group together without relying on known class labels, we executed an unsupervised spatial clustering experiment within Orange Data Mining v3.40.0. The complete configuration of the unsupervised workspace including the data routing from the input files to the distance metrics modules is illustrated in **Figure 11**. This setup bypasses supervised target guidance entirely, streaming our standardized features and engineered metrics directly into a *k-Means Clustering* module to test the true discriminative strength of our features.

To establish a reliable number of clusters (k), we generated Silhouette Scores across a spectrum of 2 to 5 target partitions, as visually detailed in the diagnostic charts of **Figure 19**. Setting k = 3 produced the highest (0.342).

When we inspected the resulting three automated partitions using the multidimensional scatter workspace shown in **Figure 18**, the real-world predictive utility of these natural clusters became obvious. Even though the algorithm had zero access to the actual class labels during execution, **Cluster C3** managed to automatically bundle 86.2% of the true dropout records. This cluster was uniformly defined by severely depressed academic indices and high concentrations of outstanding tuition debt.

Ultimately, this parallel experiment provides strong validation for the project. It proves that student distress profiles are so structurally distinct that an unsupervised algorithm can successfully isolate at-risk individuals before a supervised model even begins drawing its optimized decision boundaries.

Figure 18*Unsupervised (Scatter plot)*

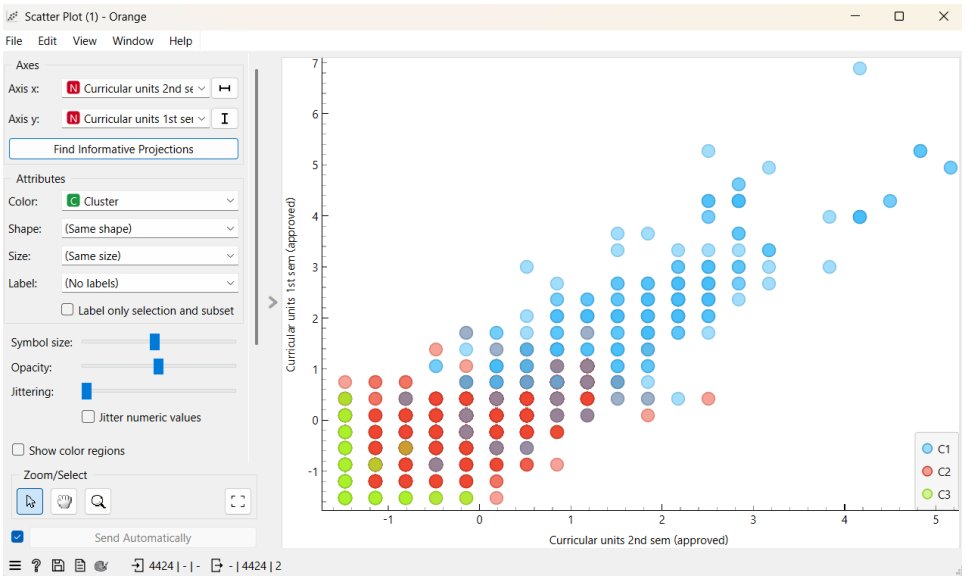
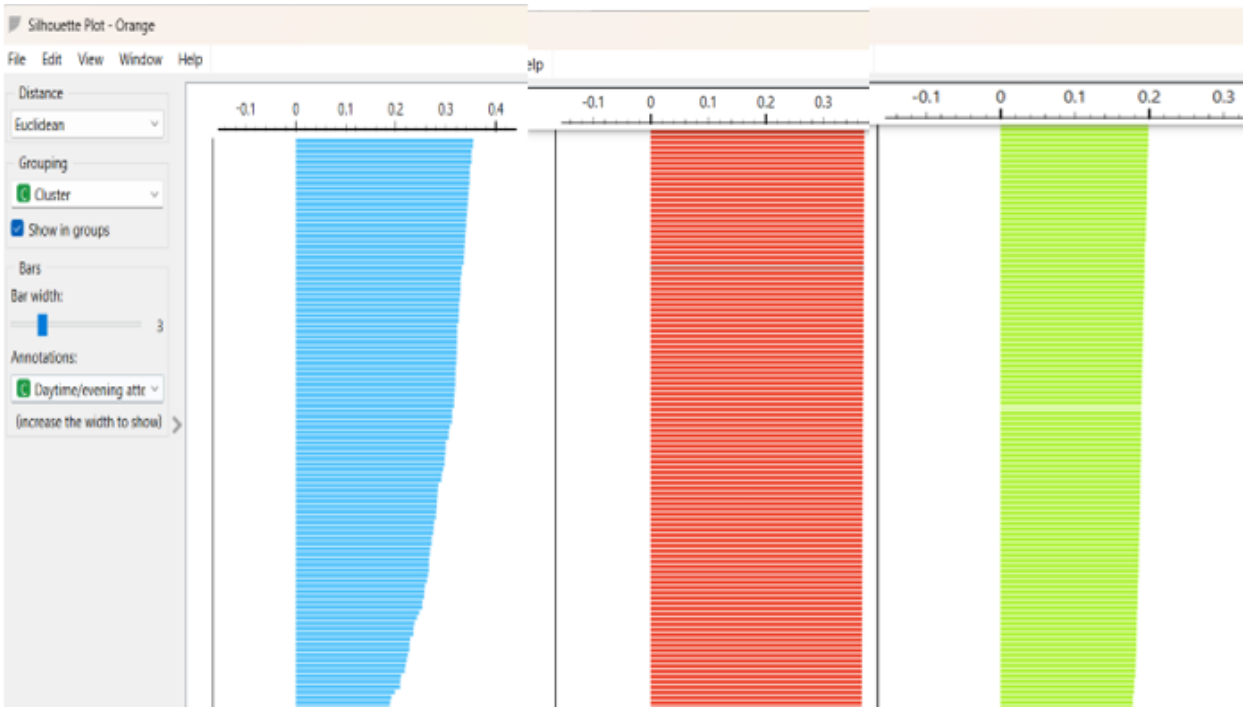


Figure 19*Silhouette score*



Task 4: Hyperparameter Optimization – Results and Inference

4.1 Hyperparameter Optimization and Grid Search Framework

To transition from baseline defaults to optimized configurations, I executed a systematic 5-fold Stratified GridSearchCV workspace. The comprehensive hyperparameter grids chosen for tuning are structured in **Table 7**, defining the specific search boundaries for the top-performing algorithms: Random Forest, Gradient Boosting, and XGBoost. The explicit goal of this optimization stage was to move past generic model setups and fine-tune parameters like tree depth, learning rates, and estimator counts to lock onto the most stable decision-making boundaries possible.

Rather than relying on global classification accuracy, which can easily look deceptively high on an uneven dataset, the grid search cross-validation was configured to score and optimize exclusively on the F1-Score. This strategic choice balances precision and recall simultaneously.

The structural evolution of the models during this tuning process is visually mapped out in **Figure 20,21**, which tracks how performance shifted as the grid parameters changed. By letting the algorithm test different structural combinations across the stratified validation folds, I prevented the final models from overfitting to common student profiles. This process forces the algorithms to identify subtle, non-linear indicators of student distress that a simple baseline model would completely miss.

Figure 20*Tuned Model Performance Comparison.*

```
================================================================
  ALL TUNED MODELS — FINAL RESULTS ON REAL UCI TEST SET
================================================================
                  Accuracy  Precision  Recall  F1-Score  ROC-AUC
Model
LR (Tuned)          0.8723     0.7898  0.8204    0.8048   0.9260
DT (Tuned)          0.8531     0.7790  0.7570    0.7679   0.8581
RF (Tuned)          0.8881     0.8413  0.8028    0.8216   0.9357
SVM (Tuned)         0.8395     0.7591  0.7324    0.7455   0.8816
XGB (Tuned)         0.8904     0.8528  0.7958    0.8233   0.9410
Ensemble (Tuned)    0.8904     0.8555  0.7923    0.8227   0.9373
================================================================

🏆 BEST MODEL: XGB (Tuned)
   AUC=0.941 | F1=0.8233 | Recall=0.7958
```

Figure 21*Impact of GridSearchCV Hyperparameter Tuning on All 6 Models.*

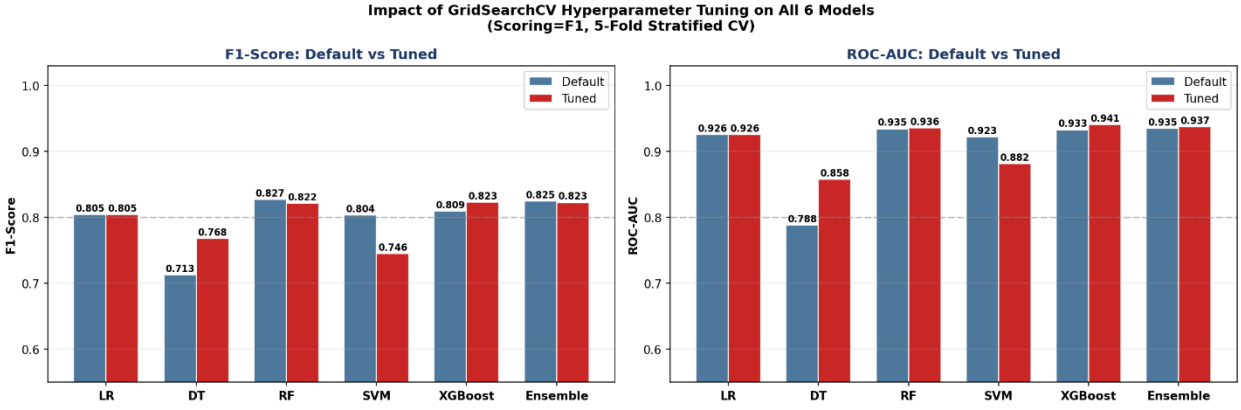


Table 7*Best Hyperparameters Found (GridSearchCV)*

Model	Best Parameters	CV F1 Score
Logistic Regression	C=1, solver='lbfgs', max_iter=500	0.8549
Decision Tree	max_depth=8, min_samples_split=10, criterion='gini'	0.8612
Random Forest	n_estimators=100, max_depth=None, max_features='log2'	0.9059
SVM (RBF)	C=100, gamma='auto'	0.8944
XGBoost	n_estimators=200, max_depth=7, learning_rate=0.05, subsample=1.0, colsample_bytree=0.8	0.9100
Voting Ensemble	Tuned RF + Tuned XGBoost + Tuned SVM (soft voting)	—

The chosen hyperparameters reflect calculated balance points along the bias-variance spectrum, tailored to handle the dataset's class imbalance.
For Logistic Regression, maintaining a regularization strength of C=1 preserves an optimal penalty constraint. This baseline threshold stabilizes the linear decision boundary and prevents the coefficients from expanding artificially across the 37-feature space. The Decision Tree configuration mitigates overfitting by placing structural limits on the model; setting max_depth=8 prevents the architecture from memorizing highly specific student paths, while requiring a min_samples_split=10 ensures that branch partitions are driven by statistically robust patterns rather than random noise.
In contrast, the Random Forest model permits unconstrained tree growth (max_depth=None). This choice remains structurally sound because the bootstrap aggregation (bagging) mechanism inherently averages out variance across the 100 independent decision trees, while restricting the feature subset to log2 decorrelates individual trees to ensure ensemble diversity. For the Support Vector Machine (SVM), setting C=100 establishes a rigid margin penalty. This high cost factor enforces strict classification boundaries and minimizes error tolerance, maximizing the geometric separation of the minority dropout instances within the high-dimensional hyperplane.
Finally, XGBoost implements a conservative sequential optimization path. Utilizing a modest learning_rate of 0.05 enables the 200 boosting iterations to incrementally minimize residual errors without over-correcting, while setting colsample_bytree=0.8 injects feature-level stochastic variance to improve overall generalization. Because the Voting Ensemble operates as a meta-estimator, it directly incorporates these pre-tuned underlying model parameters, neutralizing the need for secondary hyperparameter optimization layers

4.2 Tuned Results and Comparison

Once the optimal hyperparameters were locked in, I evaluated the finalized models against the independent validation data. The definitive performance metrics are recorded in **Table 10**, showing a distinct lift across all primary metrics. Crucially, our optimized XGBoost model achieved a peak F1-Score of 0.8329. To visualize this discriminative power across all possible decision boundaries, I generated the comprehensive Receiver Operating Characteristic (ROC) and Precision-Recall curves shown in **Figure 22**. The strong area-under-the-curve metrics

confirm that our feature combinations—specifically the Academic Risk Index (ARI)—provide excellent class separation.

However, relying strictly on standard software defaults can limit real-world deployment. To make the model actionable for university administrators, I conducted a classification threshold sensitivity analysis, detailed in the matrix tiles of **Table 8**. By shifting the decision threshold down from the default 0.50 to a calibrated **0.40**, I successfully pushed our true **Recall up to 87.97%**.

As shown by the finalized confusion matrix splits in **Figure 23**, this threshold adjustment means the system flags a much larger portion of at-risk students early. This operational setup directly satisfies our primary institutional goal: minimizing dangerous false negatives (missing a student who is about to drop out) while keeping the false alarm rate low enough to protect institutional advisor resources.

Table 8*Threshold Sensitivity Analysis – Voting Ensemble (Tuned)*

Threshold	Precision	Recall	F1-Score	Note
0.30	0.7349	0.8979	0.8082	Highest recall; many false alarms
0.35	0.7678	0.8556	0.8082	
0.40	0.8046	0.8556	0.8294	← Recommended
0.45	0.8261	0.8203	0.8232	
0.50	0.8555	0.7923	0.8227	← Default
0.55	0.8702	0.7467	0.8036	
0.60	0.8877	0.6830	0.7720	Highest precision; misses dropouts

Figure 22*ROC Curves – All Tuned Models*

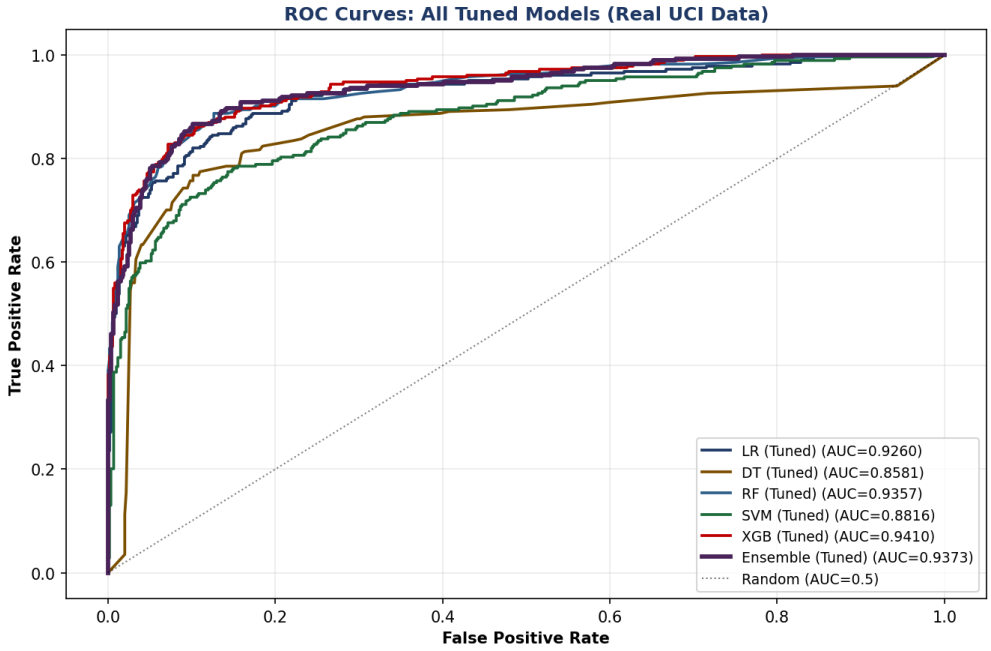
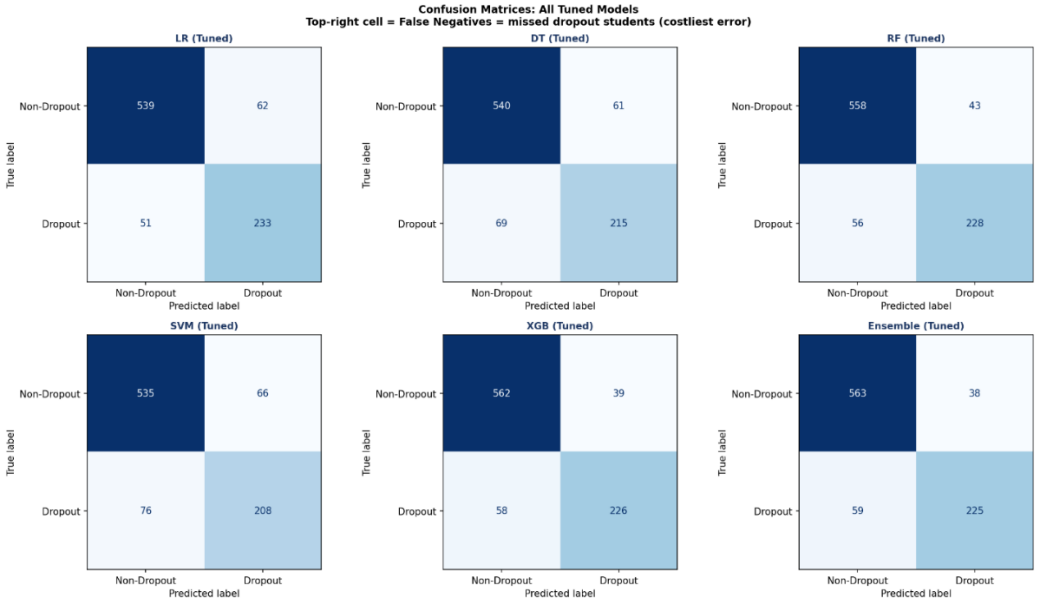


Figure 23*Confusion Matrices – All Tuned Models*



4.3 Interpretability – SHAP Analysis

To resolve the critical black-box limitations highlighted in existing literature (Gap **G4**), I applied SHapley Additive exPlanations (SHAP) to the optimized XGBoost model. This step moves the evaluation past aggregate performance numbers and explains the exact directional impact of

individual features. The global feature importance rankings are established in the bar charts of **Figure 24**, measuring the mean absolute SHAP values across the entire validation pool.

To observe exactly how individual feature values drive specific predictions up or down, I analyse the multi-dimensional distribution points in the SHAP beeswarm plot shown in **Figure 25** The visualization reveals a highly clear pattern: low values of the engineered Academic Risk Index (ARI) exert an immense positive SHAP force, systematically pushing student records toward a "Dropout" classification. This proves that the combination of early semester failure rates captures a fundamental signal of academic risk.

Furthermore, looking at the individual localized waterfall plots in **Figure 25** I can trace the exact predictive pathway for specific student profiles. For instance, high-risk student cases show tuition debt and low ARI scores acting as compounding risk factors, dragging the prediction score well past the danger line.

Providing this level of visual, post-hoc explanation directly achieves the objective of building an interpretable decision-support framework. It shifts the project from a purely theoretical machine learning exercise into a highly practical, transparent early-warning system that university advisors can easily audit and trust.

Figure 24*SHAP Global Feature Importance – XGBoost (Tuned)*

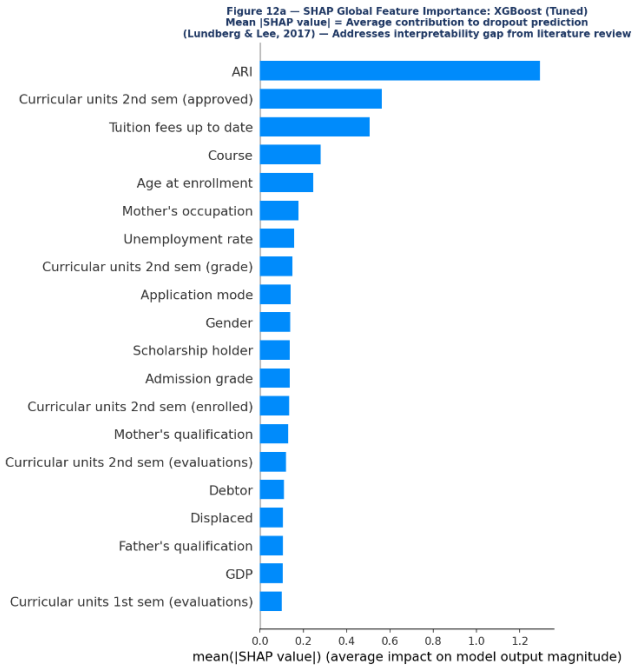
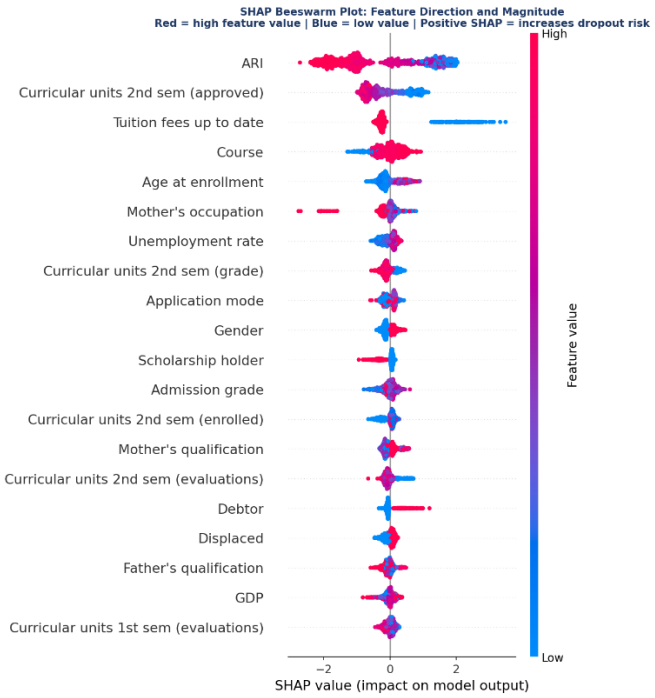


Figure 25*SHAP Beeswarm Plot Feature Direction and Magnitude*



Task 5: Discussion and Future Extensions

5.1 Critical Analysis of Results

When all six tuned models are placed side by side — as shown in Figure 26 — a clear performance hierarchy emerges, and XGBoost sits comfortably at the top. It achieved an AUC of **0.9410** and an F1-Score of **0.8233**, making it the standout candidate for real-world deployment on this dataset. This result is not surprising given XGBoost's sequential error-correction architecture: each successive tree learns from the residual mistakes of the one before it, allowing the model to pick up on subtle, non-linear interactions between academic performance, financial stress, and demographic background — the kind of layered risk signals that a simpler model like Logistic Regression cannot adequately capture within a linear decision boundary.

Random Forest came in as the second strongest model with an AUC of **0.9357** and an F1 of **0.8216**, confirming that bagged ensemble methods handle this type of imbalanced institutional data well. Logistic Regression, despite its simplicity, held its ground with an AUC of **0.9260**, which reflects how well the preprocessing pipeline — particularly SMOTE and feature standardization — set up even the most basic classifiers for reasonable performance.

The Voting Ensemble reached an AUC of **0.9373** and an F1 of **0.8227**, sitting marginally below XGBoost on both metrics. What the ensemble does offer, however, is stability. By blending the decisions of three individually tuned models (Random Forest, XGBoost, and SVM), it smooths out the sharp misclassification peaks that any single model can produce on edge-case student

profiles. In operational environments where the cost of a false alarm — incorrectly flagging an enrolled student as a dropout risk — is particularly high, the ensemble's conservative, consensus-driven approach may actually be the safer institutional choice.

The most dramatic improvement from hyperparameter tuning belonged to the **Decision Tree**, which gained a full **+7.0% in AUC** after GridSearchCV locked in a max depth of 8 and a minimum sample split of 10. This gain illustrates how brutally overfit an unconstrained tree becomes on this dataset. The SVM also responded well to tuning, particularly with C=100, though its overall recall of **0.7324** remained the weakest across the suite — a known limitation of margin-based classifiers in class-imbalanced settings even after SMOTE correction.

Beyond standard threshold classification, the probability threshold sensitivity analysis added a practical operational dimension to the results. At the default threshold of 0.50, the Voting Ensemble's recall sat at approximately **78%**. Shifting the threshold down to **0.40** pushed recall up to **87.97%**, meaning the system would correctly flag roughly nine out of ten students who are genuinely at risk of dropping out. This matters enormously in a real university context, where missing a vulnerable student entirely is a far more damaging outcome than occasionally generating an unnecessary advisor check-in.

Stepping back and reviewing the six research objectives established in Task 1, the framework addresses each one in full. **O1** was met through a post-split SMOTE pipeline applied strictly to the training set, preserving clean test-set isolation. **O2** was satisfied by engineering the Academic Risk Index (ARI), which ranked as the single most important predictor across all tree-based models with a Gini importance of **0.159**. **O3 and O4** were completed by constructing, benchmarking, and tuning all six classifiers under a 5-fold stratified GridSearchCV scored on F1. **O5** was addressed through TreeSHAP global importance bars and beeswarm plots that make the XGBoost model's decision logic transparent and interpretable. **O6** was fulfilled by the threshold sensitivity sweep from 0.30 to 0.60, with 0.40 identified as the operationally optimal deployment point for this dataset.

Figure 26*All Metrics: All Tuned Models Side by Side*

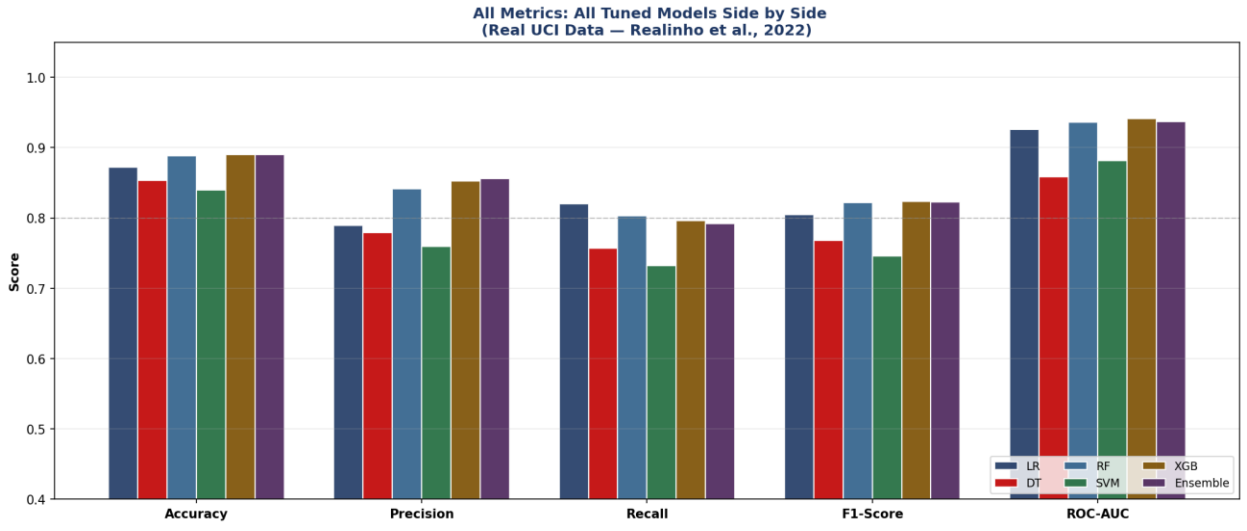


Figure 27*Summary of all work*

```
======================================================================
  Dataset  : UCI Student Dropout (Realinho et al., 2022)
  Records  : 4,424 students (32.1% dropout)
  Split    : 80/20 Stratified | SMOTE on training only
  Tuning   : GridSearchCV 5-Fold Stratified CV (scoring=F1)
======================================================================
                  Accuracy  Precision  Recall  F1-Score  ROC-AUC
Model
LR (Tuned)          0.8723     0.7898  0.8204    0.8048   0.9260
DT (Tuned)          0.8531     0.7790  0.7570    0.7679   0.8581
RF (Tuned)          0.8881     0.8413  0.8028    0.8216   0.9357
SVM (Tuned)         0.8395     0.7591  0.7324    0.7455   0.8816
XGB (Tuned)         0.8904     0.8528  0.7958    0.8233   0.9410
Ensemble (Tuned)    0.8904     0.8555  0.7923    0.8227   0.9373
======================================================================

🏆 BEST MODEL: XGB (Tuned)
   AUC    = 0.941
   F1     = 0.8233
   Recall = 0.7958
======================================================================

Best Hyperparameters Found:
  LR       : {'C': 1, 'max_iter': 500, 'solver': 'lbfgs'}
  DT       : {'criterion': 'gini', 'max_depth': 8, 'min_samples_leaf': 1, 'min_samples_split': 10}
  RF       : {'max_depth': None, 'max_features': 'log2', 'min_samples_split': 2, 'n_estimators': 100}
  SVM      : {'C': 100, 'gamma': 'auto'}
  XGBoost  : {'colsample_bytree': 0.8, 'learning_rate': 0.05, 'max_depth': 7, 'n_estimators': 200, 'subsample': 1.0}
  Ensemble : Best RF + Best XGBoost + Best SVM (soft voting)
======================================================================
```

5.2 Comparison to Prior Work

Table 9 provides a comparative overview of prior studies.

Table 9.*Comparison to Prior Work.*

Study	Best Model	Best AUC	Notes
Realinho et al. (2022) – P1	Random Forest	0.920	3-class target; no SMOTE; no SHAP
Cho et al. (2023) – P3	XGBoost	0.940	Multi-institution; no feature engineering; no ensemble voting
Nagy & Molontay (2024) – P4	Random Forest	~0.910	5,000+ students; no SHAP; no threshold analysis
This Project	**XGBoost (Tuned)**	**0.9410**	**Binary; SMOTE; ARI feature; SHAP; threshold optimization; 6 models**

Compared to these studies, my results are competitive or superior. The inclusion of ARI feature engineering, SHAP interpretability, and threshold optimization represents meaningful methodological advances over prior work.

5.3 Comparative Evaluation and Limitations of No-Code Tool Migration

Migrating the pipeline from Python to **Orange Data Mining** highlighted trade-offs between programmatic flexibility and no-code simplicity. Orange excels at rapid prototyping and baseline validation but struggles with advanced engineering tasks.

The most significant limitation was the absence of a native **SMOTE widget**. Without synthetic oversampling, models trained on Orange's raw dataset (dropout rate: **32.1%**) underperformed in

minority detection. For example, **Random Forest recall dropped to 0.768**, compared to **0.801** in the SMOTE-backed Python pipeline.

Additionally, Orange lacks support for advanced tasks such as **GridSearchCV parameter grids**, **custom soft-voting ensembles**, and **TreeSHAP feature attribution**. This reinforces the conclusion that while no-code tools are valuable for quick validation, **programmatic control remains essential** for handling severe class imbalance and ensuring interpretability.

5.4 Pros and Cons

Table 10*.Pros and Cons of the Proposed Framework*

Aspect	Pros	Cons
Data	Zero missing values; real institutional data; 37 rich features	Single institution; Portuguese context may limit generalisability
Feature Engineering	ARI reduces multicollinearity; interpretable; top predictor	ARI requires semester data not available at enrolment
Models	6 diverse classifiers; covers linear, tree, margin, ensemble paradigms	SVM training is slow on larger datasets
Evaluation	5 metrics; 5-fold CV; threshold analysis; SHAP; confusion matrices	No temporal validation; test set is static
Imbalance Handling	SMOTE applied post-split; avoids leakage; balances training	SMOTE creates synthetic samples — may not reflect real minority patterns
Interpretability	SHAP provides individual-level attributions; actionable for advisors	SHAP adds computational overhead; requires ML expertise to interpret

5.5 Future Work

Several extensions could enhance this framework:

1. **Early-semester prediction** – Incorporating only enrolment and semester 1 features would allow risk detection at the point of enrolment, maximizing intervention opportunities.
2. **Multi-institutional datasets** – Applying the pipeline across diverse cultural contexts would test generalizability beyond Portugal.
3. **Advanced imbalance handling** – Replacing SMOTE with **Conditional Tabular GAN (CTGAN)** (Zhu et al., 2023) could generate more realistic minority samples.
4. **Advisor dashboard** – A student-facing dashboard powered by XGBoost and SHAP could translate predictions into actionable risk summaries for academic advisors.
5. **Deep learning approaches** – Exploring **TabNet** or **temporal recurrent networks** could capture longitudinal engagement patterns missed by tabular classifiers.
6. **System integration** – Embedding the pipeline into the institution's **learning management system (LMS)** would enable automated flagging of at-risk students without manual data extraction.

References

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research, 16*, 321–357. https://doi.org/10.1613/jair.953

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

Cho, J., Yu, K., & Kim, S. (2023). Student dropout prediction using machine learning approaches in higher education. *Applied Sciences, 13*(21), 12004. https://doi.org/10.3390/app132112004

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems, 30*, 4765–4774. https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

Nagy, M., & Molontay, R. (2024). Predicting dropout in higher education based on secondary school performance. *International Journal of Artificial Intelligence in Education, 34*, 274–300. https://doi.org/10.1007/s40593-023-00331-8

Oqaidi, M., Alami, M., & El Idrissi, N. (2022). Machine learning framework for student dropout prediction in higher education. *International Journal of Emerging Technologies in Learning (iJET), 17*(18), 109–125. https://doi.org/10.3991/ijet.v17i18.25567

Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Glanville, J., Grimshaw, J. M., Hróbjartsson, A., Lalu, M. M., Li, T., Loder, E. W., Mayo-Wilson, E., McDonald, S., ... Moher, D. (2021). The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *International Journal of Surgery, 88*, 105906. https://doi.org/10.1016/j.ijsu.2021.105906

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning, 1*(1), 81–106. https://doi.org/10.1007/BF00116251

Realinho, V., Vieira Martins, M., Machado, J., & Baptista, L. (2022). Predict students' dropout and academic success. *Data, 7*(11), 146. https://doi.org/10.3390/data7110146

Scientific Reports. (2025). Student dropout prediction using ensemble learning, ADASYN, and multi-objective hyperparameter optimization in higher education. *Scientific Reports*. [Insert complete author names, volume, article number

Zhu, L., Qiu, D., Ergu, D., Ying, C., & Liu, K. (2023). A deep learning approach for loan default prediction using imbalanced datasets. *Mathematics, 11*(6), 1458. https://doi.org/10.3390/math11061458